INSTITUTO TECNOLOGICO NACIONAL DE MEXICO

CAMPUS CULIACAN

INGENIERIA EN SISTEMAS COMPUTACIONALES

MATERIA

TALLER DE SISTEMAS OPERATIVOS

TEMA

PROCESAMIENTO DE ARCHIVO TXT

DOCENTE

ZURIEL DATHAN MORA FELIX

ELABORADO POR

JESUS IVETH RIVERA LUNA

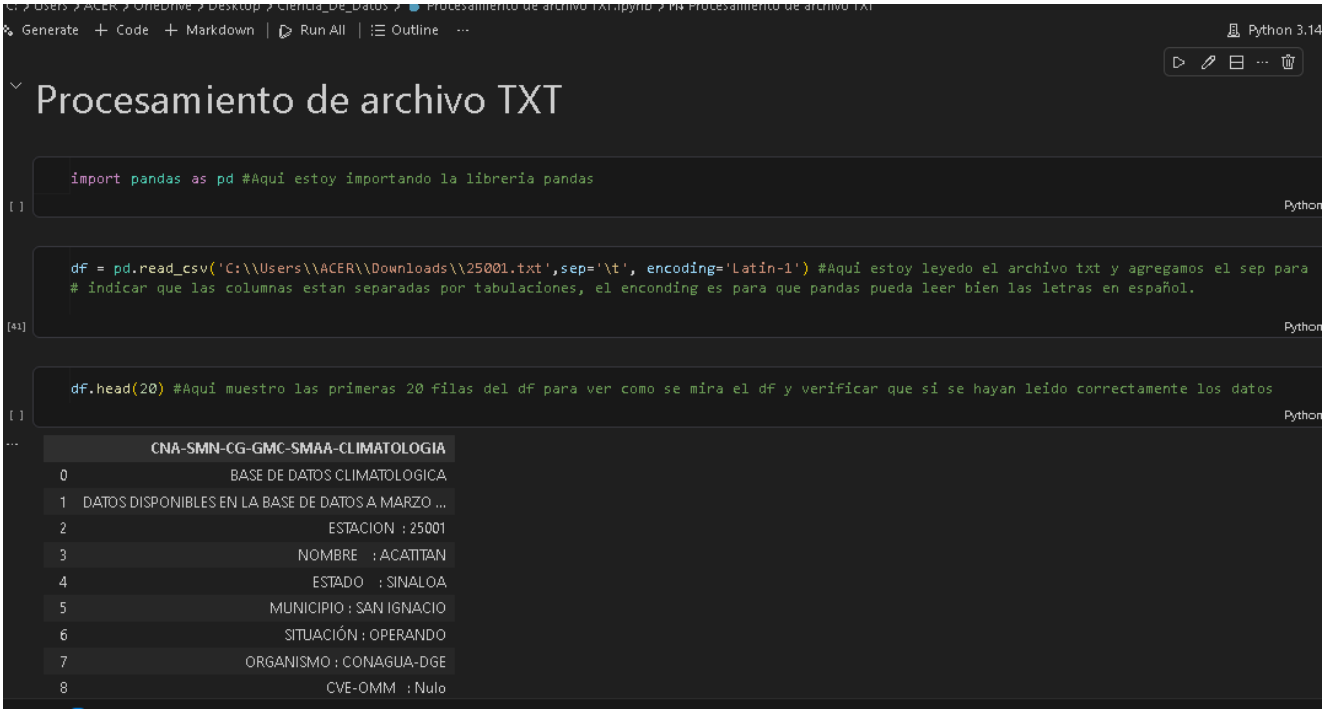
HORA

12:00 – 01:00 PM

Culiacán, Sinaloa 31/03/2026

Link: https://github.com/IvethRivera/Ciencia-De-Datos-phyton/blob/main/Procesamiento%20de%20archivo%20TXT.ipynb?short_path=8c2b076

En esta captura importé la librería pandas y leí el archivo 25001.txt con pd.read_csv, usando sep='\t' y encoding='Latin-1' para que reconociera las letras en español. También usé df.head(20) para mostrar las primeras 20 filas y verificar que los datos se leyeron correctamente.



Aquí intenté mostrar las columnas con df.columns pero noté que los datos no se leyeron bien, solo aparecía 1 columna. Agregué el parámetro skiprows=18 para saltar las primeras filas que eran encabezado, y con df.info() pude observar que aún había solo 1 columna, lo que indicaba que las columnas no estaban correctamente separadas.

```python
df.columns # aqui intento mostrar las columnas de los datos a analizar, pero no se leyeron de forma correcta
```

```
Index(['CNA-SMN-CG-GMC-SMAA-CLIMATOLOGIA'], dtype='str')
```

```python
df = pd.read_csv('C:\\Users\\ACER\\Downloads\\25001.txt',encoding='Latin-1', skiprows=18) # aqui agregamos el skiprows que es un parametro que
#nos sirve para saltanos filas
```

```python
df.columns # ya podemos ver que se muestra la columna de los datos
```

```
Index([' FECHA     (MM)   (MM)   (°C)   (°C)'], dtype='str')
```

```python
df.info() #con el df veremos la informacion de los datos donde podemos observar un errordonde solo esta contando 1 columna y deberian ser 5
#muestra todo el encabezadodel df y lo lee como texto puede ser que no esta separado por tabulaciones
```

```
<class 'pandas.DataFrame'>
RangeIndex: 19575 entries, 0 to 19574
Data columns (total 1 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0    FECHA     (MM)   (MM)   (°C)   (°C)  19575 non-null  str
dtypes: str(1)
memory usage: 153.1 KB
```

En esta captura usé with open para explorar la estructura del archivo y ver cómo estaba separado. Observé que las líneas del 0 al 17 son datos del encabezado (nombre de estación, estado, municipio, etc.). También cambié el separador a sep=r"\s+" para que reconociera los espacios en blanco como separadores.

```python
with open ('C:\\Users\\ACER\\Downloads\\25001.txt',encoding='Latin-1') as f: #este codigo nos ayuda a ver como es la estructura del archivo
    for i, linea in enumerate(f):                                            # y porque esta separado el archivo
        print(i, repr(linea))
        if i > 16: break
```

```
0 'CNA-SMN-CG-GMC-SMAA-CLIMATOLOGIA\n'
1 'BASE DE DATOS CLIMATOLOGICA\n'
2 'DATOS DISPONIBLES EN LA BASE DE DATOS A MARZO 2020;CON LA INFORMACION SUMINISTRADA POR LAS OFICINAS REGIONALES\n'
3 ' \n'
4 'ESTACION  : 25001\n'
5 'NOMBRE    : ACATITAN\n'
6 'ESTADO    : SINALOA\n'
7 'MUNICIPIO : SAN IGNACIO\n'
8 'SITUACIÓN : OPERANDO\n'
9 'ORGANISMO : CONAGUA-DGE\n'
10 'CVE-OMM   : Nulo\n'
11 'LATITUD   : 024.097°\n'
12 'LONGITUD  : -106.667°\n'
13 'ALTITUD   : 96 msnm\n'
14 ' \n'
15 'EMISION   : 06/04/2020 \n'
16 ' \n'
17 '           PRECIP  EVAP   TMAX   TMIN\n'
```

```python
df = pd.read_csv('C:\\Users\\ACER\\Downloads\\25001.txt',encoding='Latin-1', skiprows=18, sep=r"\s+",
    engine="python",) # para una mejor lectura del archivo cambiamos lo del sep para indicar que las columnas estan separadas por espacios
    # en blancos
```

Ya con el separador correcto, las columnas se mostraron bien: FECHA, PRECIP, EVAP, TMAX, TMIN. Las renombré con df.columns =

["FECHA","PRECIP","EVAP","TMAX","TMIN"] para que fueran más fáciles de entender, y usé df.head() para visualizar las primeras filas del dataframe.

```
df = pd.read_csv('C:\\Users\\ACER\\Downloads\\25001.txt',encoding='Latin-1', skiprows=18, sep=r"\s+",
    engine="python",) # para una mejor lectura del archivo cambiamos lo del sep para indicar que las columnas estan separadas por espacios
    # en blancos
```

```
print(df.columns) # aqui podemos ver que las columnas se miran, pero las cambiaremos a unas mas facil de comprender
```

```
Index(['FECHA', '(MM)', '(MM).1', '(°C)', '(°C).1'], dtype='str')
```

```
df.columns = ["FECHA","PRECIP","EVAP","TMAX","TMIN"] #ya podemos ver que yase muestran las columnas con nombres faciles de entender
```

```
df.head() # se muestra el df
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
0	01/01/1961	0	Nulo	25	13
1	02/01/1961	0	Nulo	27.5	14
2	03/01/1961	0	Nulo	31	11
3	04/01/1961	0	Nulo	32	13
4	05/01/1961	0	Nulo	29	14

Verifiqué con df.columns que los nombres estaban correctos, con df.info() confirmé que ya había 5 columnas con 19,575 entradas, y con df.shape se mostró que el dataframe tenía 19,575 filas y 5 columnas.

```
df.columns # podemos observarlas columnas cambiadas
```

```
Index(['FECHA', 'PRECIP', 'EVAP', 'TMAX', 'TMIN'], dtype='str')
```

```
df.info() # se mira la informacion del df y ya se muestra que si hay 5 columnas y no 1
```

```
<class 'pandas.DataFrame'>
RangeIndex: 19575 entries, 0 to 19574
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   FECHA   19575 non-null  str
 1   PRECIP  19574 non-null  str
 2   EVAP    19574 non-null  str
 3   TMAX    19574 non-null  str
 4   TMIN    19574 non-null  str
dtypes: str(5)
memory usage: 764.8 KB
```

```
df.shape # con el shape que son 5 columnas y 19575 filas
```

```
(19575, 5)
```

Usé df.describe() para ver el resumen estadístico del dataframe. Con df.isnull().sum() revisé si había valores nulos (había 1 por columna), y con (df == "Nulo").sum().sum() encontré que había 323 valores marcados como "Nulo" en los datos.

```
df.describe() # podemos obervar el resumen estidstico
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
count	19575	19574	19574	19574	19574
unique	19575	493	135	60	63
top	01/01/1961	0	3	35	23
freq	1	16087	401	1980	1225

```
df.isnull().sum() # con el isnull sum podemos ver si hay datos faltantes y ya se muestra que no hay datos faltanes en el df
```

```
FECHA     0
PRECIP    1
EVAP      1
TMAX      1
TMIN      1
dtype: int64
```

```
( df == "Nulo").sum().sum() #podemos obserbar que si hay datos faltantes como  nulos
```

```
np.int64(323)
```

Con df.dtypes verifiqué los tipos de datos de cada columna (todas estaban como str). Con df.index comprobé que el índice iba del 0 al 19,574. Guardé las columnas en una variable lista_colum y usé df.tail() para ver los últimos datos del dataframe.

```
df.dtypes # podemos ver los tipos de datos y ya se muestra que la fecha es un objeto y las demas columnas enteras
```

```
FECHA     str
PRECIP    str
EVAP      str
TMAX      str
TMIN      str
dtype: object
```

```
df.index # con el index vemos que es un rango del 0 al 19574
```

```
RangeIndex(start=0, stop=19575, step=1)
```

```
lista_colum = df.columns # con el codigo de arriba guardamos las columnas del df con una variable
```

```
lista_colum # aqui mostramos la variable que guardamos en las columna del df
```

```
Index(['FECHA', 'PRECIP', 'EVAP', 'TMAX', 'TMIN'], dtype='str')
```

```
df.tail() # aqui podemos observar los ultimos datos
```

on df.tail() observé los últimos registros del dataframe, donde aparecían valores NaN en la última fila. Luego creé la función cob_nuloanan(df) usando df.replace("Nulo", np.nan) para convertir los valores "Nulo" en valores NaN reales. Al verificar con (df == "Nulo").sum().sum() el resultado fue 0, confirmando que ya no había valores "Nulo".

```python
df.tail() # aqui podemos observar los ultimos datos
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
19570	28/07/2018	9.8	3.7	37	23.5
19571	29/07/2018	6	3.3	35	20
19572	30/07/2018	48	6	36	22
19573	31/07/2018	2	3.7	34	22
19574	----------	NaN	NaN	NaN	NaN

```python
import numpy as np # importamos la libreria numpy para poder utilizar el np.nan
def cob_nuloanan(df):
    df.replace("Nulo", np.nan, inplace=True) #aqui quitamos los valores nulos
```

```python
cob_nuloanan(df)
```

```python
(df == "Nulo").sum().sum() #verificamos que no ya no hay valores nulos
```

```
np.int64(0)
```

```python
def camb_valores(df): # esta funcion nos ayuda a cmabiar los valores de las columnas a numeros , con el errors='coerce', y manejar error con nan
    columnas = ["PRECIP","EVAP","TMAX","TMIN"]
    for col in columnas:
        df[col] = pd.to_numeric(df[col], errors='coerce')
```

Python

```python
camb_valores(df)
```

Python

```python
df.info() # entendemos como estan organizados los datos y si hay valores faltantes
```

Python

```
<class 'pandas.DataFrame'>
RangeIndex: 19575 entries, 0 to 19574
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   FECHA   7720 non-null   datetime64[us]
 1   PRECIP  19575 non-null  float64
 2   EVAP    19575 non-null  float64
 3   TMAX    19575 non-null  float64
 4   TMIN    19575 non-null  float64
dtypes: datetime64[us](1), float64(4)
memory usage: 764.8 KB
```

```python
def camb_fecha(df):
    df["FECHA"] = pd.to_datetime(df["FECHA"], dayfirst=True, errors='coerce') # se ponen los valores como nulos que esten  mal escritos
    #o no sean feachas validas
```

```python
camb_fecha(df)
```

```python
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 19575 entries, 0 to 19574
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   FECHA   7720 non-null   datetime64[us]
 1   PRECIP  19575 non-null  float64
 2   EVAP    19575 non-null  float64
 3   TMAX    19575 non-null  float64
 4   TMIN    19575 non-null  float64
dtypes: datetime64[us](1), float64(4)
memory usage: 764.8 KB
```

```python
df.head()
```

```python
df.head()
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
0	1961-01-01	0.0	4.882157	25.0	13.0
1	1961-02-01	0.0	4.882157	27.5	14.0
2	1961-03-01	0.0	4.882157	31.0	11.0
3	1961-04-01	0.0	4.882157	32.0	13.0
4	1961-05-01	0.0	4.882157	29.0	14.0

```python
df["FECHA"].isnull().sum() #aqui reviso cunatas fechas vacias tiene mi tabla, habias bastantes huecos
```

```
np.int64(11855)
```

```python
df = df.dropna(subset=["FECHA"]) #aqui elimino las las filas donde la fecha esta vacia
```

```python
df.isnull().sum() #reviso si aun quedan valores vacios, y veo el numero de columnas y a lo que se reducieron los datos
df.shape
```

```
(7720, 5)
```

```python
df.fillna(df.mean(numeric_only=True), inplace=True) #aqui relleno los nulos con el promedio de cada columna
#por eso algunos salen repetidos
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
0	1961-01-01	0.0	4.882157	25.0	13.0
1	1961-02-01	0.0	4.882157	27.5	14.0
2	1961-03-01	0.0	4.882157	31.0	11.0
3	1961-04-01	0.0	4.882157	32.0	13.0
4	1961-05-01	0.0	4.882157	29.0	14.0
...	...	...	...	...	...
19550	2018-08-07	4.4	4.600000	34.0	19.5
19551	2018-09-07	0.0	1.400000	32.5	22.0
19552	2018-10-07	2.0	3.200000	32.0	24.0
19553	2018-11-07	0.0	0.800000	35.0	21.0
19554	2018-12-07	0.0	4.700000	36.0	20.5

7720 rows × 5 columns

```python
df.isnull().sum().sum() #aqui reviso si aun quedaron valos vacios, me dio como resultado 0 eso quiere decir que ya no hay
```

```
np.int64(0)
```

```python
df.mean(numeric_only=True) #me calcula el promedio de cada columna numerica
df.std(numeric_only=True) # me calcula la desviacion estandar, es decir que tan dsipersos estan los datos respecto al promedio
df.describe() #hace un resumen completo de la tabla
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
count	7720	7720.000000	7720.000000	7720.000000	7720.000000
mean	1989-02-27 11:30:09.326424	2.227830	4.850822	34.451848	16.600287
min	1961-01-01 00:00:00	0.000000	0.000000	18.500000	-2.500000
25%	1974-05-10 18:00:00	0.000000	3.200000	32.500000	11.000000
50%	1987-11-08 12:00:00	0.000000	4.600000	35.000000	16.000000
75%	2004-04-06 06:00:00	0.000000	6.325000	37.000000	23.000000
max	2018-12-07 00:00:00	205.400000	14.300000	44.500000	28.500000
std	NaN	8.933674	2.248694	3.403526	6.245972

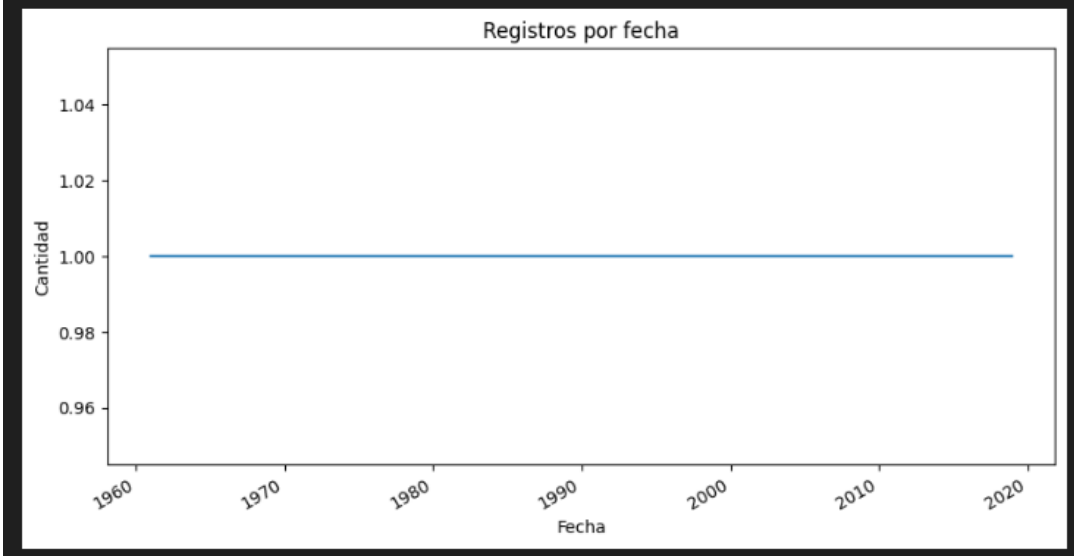
```python
df.groupby("FECHA").size() #agrupa los datos por fecha y cuantas filas hay por cada dia me muestra que 1 eso quiere decir que no hay repetidas
```

```
FECHA
1961-01-01    1
1961-01-02    1
1961-01-03    1
1961-01-04    1
1961-01-05    1
             ..
2018-12-02    1
2018-12-03    1
```

```
2018-12-02    1
2018-12-03    1
2018-12-04    1
2018-12-06    1
2018-12-07    1
Length: 7720, dtype: int64
```

```python
df.isnull().sum() # cuentos los valores vacios de cada columna
df[df.isnull().any(axis=1)] #aqui me muestra todas las filas que tengan unnvalor vacio
```

	FECHA	PRECIP	EVAP	TMAX	TMIN
12	NaT	0.000000	4.882157	32.000000	10.00000
13	NaT	0.000000	4.882157	31.000000	10.00000
14	NaT	0.000000	4.882157	32.000000	8.50000
15	NaT	0.000000	4.882157	31.000000	11.00000
16	NaT	0.100000	4.882157	28.000000	14.50000
...	...	...	...	...	...
19570	NaT	9.800000	3.700000	37.000000	23.50000
19571	NaT	6.000000	3.300000	35.000000	20.00000
19572	NaT	48.000000	6.000000	36.000000	22.00000
19573	NaT	2.000000	3.700000	34.000000	22.00000
19574	NaT	2.217452	4.882157	34.468808	16.65742

11855 rows × 5 columns

```
df.groupby("FECHA").size().plot(figsize=(10,5)) #como extra cree una grafica, podemos obserbar que no hay valores repetidos
plt.title("Registros por fecha")
plt.xlabel("Fecha")
plt.ylabel("Cantidad")
plt.show()
```